Nama: ALDI SAPUTRA

NIM: 105841115923

Pertemuan 05: Support Vector Machine (SVM)

Tujuan Praktikum: Pada pertemuan kelima ini, saya mempraktikkan cara kerja algoritma Support Vector Machine (SVM) untuk kasus klasifikasi dua kelas (biner). Target utamanya adalah memahami pentingnya merapikan skala data (*scaling*), melihat perbedaan kinerja antara kernel Linear dan RBF, serta mencoba mencari pengaturan parameter terbaik (*hyperparameter tuning*) secara manual.

Persiapan Data dan Scaling

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load & Split Dataset
# ==========================================================
# Menggunakan Breast Cancer Wisconsin dataset (Klasifikasi Biner: Malignant/Benign)
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

#

# Scaling (Wajib untuk SVM)
# SVM mencari margin maksimum antar kelas. Jika skala fitur berbeda (misal: 0-1 vs 0-1000),
# fitur dengan skala besar akan mendominasi perhitungan jarak.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Langkah pertama yang saya lakukan adalah memuat dataset *Breast Cancer Wisconsin* dari library Scikit-learn, yang berisi data medis untuk membedakan tumor ganas (*Malignant*) dan jinak (*Benign*). Data ini kemudian saya bagi menjadi dua bagian: 80% untuk melatih model (*training*) dan 20% untuk pengujian (*testing*).

Satu hal yang sangat krusial sebelum memasukkan data ke model SVM adalah melakukan standardisasi menggunakan StandardScaler. Hal ini wajib dilakukan karena algoritma SVM sangat bergantung pada perhitungan jarak (*Euclidean distance*) antar titik data. Jika skala antar

fitur berbeda terlalu jauh (misalnya ada fitur bernilai ribuan dan ada yang hanya desimal), modelnya akan menjadi bias terhadap angka yang besar. Oleh karena itu, semua datanya harus diseragamkan terlebih dahulu skalanya.

Tugas 1: Membandingkan Kernel Linear dan RBF

```
# Tugas 1: Bandingkan Kernel Linear vs RBF
# ==========================================================
# Kernel trick memungkinkan SVM bekerja pada ruang dimensi tinggi tanpa menghitung koordinat eksplisit
models = {
    "SVM Linear": SVC(kernel="linear", C=1),
    "SVM RBF": SVC(kernel="rbf", C=1, gamma="scale"),
}

print("-" * 60)
print("Tugas 1: Perbandingan Kernel Linear vs RBF")
print("-" * 60)

#

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = round(accuracy_score(y_test, pred), 3)
    results[name] = acc
    print(f"\n{name} Accuracy: {acc}")
    print(f"Confusion Matrix {name}:")
    print(confusion_matrix(y_test, pred))
    print(f"Classification Report {name}:")
    print(classification_report(y_test, pred, target_names=data.target_names))
```

Di bagian ini, saya bereksperimen dengan dua jenis batas pemisah (kernel) pada SVM, yaitu Linear dan RBF.

```
...  ------------------------------------------------------------
     Tugas 1: Perbandingan Kernel Linear vs RBF
     ------------------------------------------------------------

     SVM Linear Accuracy: 0.956
     Confusion Matrix SVM Linear:
     [[41  2]
      [ 3 68]]
     Classification Report SVM Linear:
                   precision    recall  f1-score   support

        malignant       0.93      0.95      0.94        43
           benign       0.97      0.96      0.96        71

         accuracy                           0.96       114
        macro avg       0.95      0.96      0.95       114
     weighted avg       0.96      0.96      0.96       114


     SVM RBF Accuracy: 0.982
     Confusion Matrix SVM RBF:
     [[41  2]
      [ 0 71]]
     Classification Report SVM RBF:
                   precision    recall  f1-score   support

        malignant       1.00      0.95      0.98        43
           benign       0.97      1.00      0.99        71

         accuracy                           0.98       114
        macro avg       0.99      0.98      0.98       114
     weighted avg       0.98      0.98      0.98       114
```

- **Kernel Linear** bertugas membuat batas keputusan berupa garis atau bidang lurus.
- **Kernel RBF** (*Radial Basis Function*) lebih fleksibel karena bisa menyesuaikan batas pemisahnya agar melengkung mengikuti pola data yang kompleks atau tidak beraturan.

Kedua model ini kemudian dilatih dan dites akurasinya untuk melihat pendekatan mana yang lebih cocok untuk membedakan kategori tumor pada dataset ini.

Tugas 2: Tuning Parameter Manual (C dan Gamma)

```python
# Tugas 2: Tuning Sederhana (C dan Gamma)
# ==========================================================
# C: Kontrol trade-off antara margin lebar dan klasifikasi yang benar (Regularisasi).
# Gamma: Menentukan seberapa jauh pengaruh satu contoh pelatihan mencapai (Hanya untuk RBF/Poly).
print("\n" + "=" * 60)
print("Tugas 2: Parameter Tuning Analysis (C dan Gamma)")
print("=" * 60)

#

best_acc = 0
best_params = {}

# Mencari kombinasi terbaik secara manual (mirip Grid Search sederhana)
for c_val in [0.1, 1, 10]:
    for gamma_val in [0.01, 0.1, 1]:
        model = SVC(kernel="rbf", C=c_val, gamma=gamma_val)
        model.fit(X_train, y_train)
        y_pred_tune = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred_tune)

        print(f"  C={c_val:>4}, gamma={gamma_val:>4} → Accuracy: {acc:.3f}")

        if acc > best_acc:
            best_acc = acc
            best_params = {"C": c_val, "gamma": gamma_val}

print(f"\n→ Parameter Terbaik Hasil Tuning: C={best_params['C']}, gamma={best_params['gamma']}")
print(f"→ Akurasi Tertinggi: {best_acc:.3f}")
```

Selain membandingkan kernel, saya juga ditugaskan untuk menguji coba kombinasi parameter terbaik. Alih-alih menggunakan fungsi otomatis yang sudah ada, saya merancang perulangan (*loop*) secara manual untuk menguji berbagai kombinasi nilai parameter **C** (0.1, 1, 10) dan parameter **Gamma** (0.01, 0.1, 1).

```
...
    ============================================================
    Tugas 2: Parameter Tuning Analysis (C dan Gamma)
    ============================================================
      C= 0.1, gamma=0.01 → Accuracy: 0.965
      C= 0.1, gamma= 0.1 → Accuracy: 0.947
      C= 0.1, gamma=   1 → Accuracy: 0.623
      C=   1, gamma=0.01 → Accuracy: 0.965
      C=   1, gamma= 0.1 → Accuracy: 0.965
      C=   1, gamma=   1 → Accuracy: 0.632
      C=  10, gamma=0.01 → Accuracy: 0.982
      C=  10, gamma= 0.1 → Accuracy: 0.965
      C=  10, gamma=   1 → Accuracy: 0.632

    → Parameter Terbaik Hasil Tuning: C=10, gamma=0.01
    → Akurasi Tertinggi: 0.982
```

- **Parameter C** berfungsi untuk mengatur seberapa ketat model menoleransi kesalahan klasifikasi. Nilai C yang besar akan membuat batas margin menjadi lebih sempit.
- **Parameter Gamma** menentukan seberapa jauh jangkauan pengaruh dari masing-masing titik sampel data.

Program akan menjalankan simulasi pada setiap kombinasi tersebut dan mencatat setelan mana yang menghasilkan akurasi pemodelan paling tinggi.

Kesimpulan dan Interpretasi (Tugas 3)

```python
# Tugas 3: Kesimpulan & Interpretasi
# ============================================================
print("\n" + "=" * 60)
print("Tugas 3: Kesimpulan")
print("=" * 60)
print("1. Pentingnya Scaling: Tanpa StandardScaler, performa SVM akan menurun drastis")
print("   karena algoritma ini berbasis perhitungan jarak (Euclidean).")
print(f"2. Efektivitas Kernel: {'SVM RBF' if results['SVM RBF'] >= results['SVM Linear'] else 'SVM Linear'}")
print("   cenderung memberikan hasil lebih baik karena mampu menangkap pola non-linear.")
print("3. Pengaruh Parameter:")
print(f"   - Nilai C yang terlalu besar dapat menyebabkan overfitting (margin sempit).")
print(f"   - Nilai Gamma yang terlalu tinggi membuat model terlalu sensitif pada data point terdekat.")
print(f"4. Rekomendasi: Gunakan C={best_params['C']} dan Gamma={best_params['gamma']} untuk model produksi.")

print("\n✅ Pertemuan 05 selesai.")
```

```
============================================================
Tugas 3: Kesimpulan
============================================================
1. Pentingnya Scaling: Tanpa StandardScaler, performa SVM akan menurun drastis
   karena algoritma ini berbasis perhitungan jarak (Euclidean).
2. Efektivitas Kernel: SVM RBF
   cenderung memberikan hasil lebih baik karena mampu menangkap pola non-linear.
3. Pengaruh Parameter:
   - Nilai C yang terlalu besar dapat menyebabkan overfitting (margin sempit).
   - Nilai Gamma yang terlalu tinggi membuat model terlalu sensitif pada data point terdekat.
4. Rekomendasi: Gunakan C=10 dan Gamma=0.01 untuk model produksi.

✅ Pertemuan 05 selesai.
```

Dari pengerjaan kode di atas, saya merangkum beberapa kesimpulan dan *insight* penting:

1. **Pentingnya Scaling:** Jika kita lupa menggunakan StandardScaler, performa klasifikasi model SVM akan langsung menurun drastis karena sistem penentuan jaraknya tidak berjalan dengan baik.

2. **Efektivitas Kernel:** Secara umum, model dengan kernel RBF memberikan hasil klasifikasi yang lebih baik dibandingkan Linear. Ini membuktikan bahwa pola persebaran data medis di dunia nyata cenderung non-linear dan butuh pemisah yang lebih luwes.

3. **Risiko Overfitting:** Saat mengatur parameter, kita harus berhati-hati. Jika kita memasukkan nilai **C** yang terlalu besar, model akan memaksakan margin yang terlalu sempit sehingga berisiko *overfitting*. Begitu juga dengan **Gamma**; jika angkanya terlalu tinggi, model menjadi sangat sensitif dan hanya memedulikan titik data yang paling dekat dengannya.

4. **Rekomendasi Terbaik:** Berdasarkan hasil pengujian (*tuning*) manual yang dilakukan oleh program, kombinasi paling optimal untuk model produksi pada kasus ini adalah dengan menggunakan nilai **C=10** dan **Gamma=0.01**.